PROYECTO ALKE WALLET

MODULO 6

NOMBRE: Felipe Leyton

1. Arquitectura MVVM (Model-View-ViewModel)

Para este proyecto, separaremos las responsabilidades para asegurar un código mantenible y testeable:

Model: Clases de datos (POJOs) y las entidades de Room.

Repository: Clase que decide si los datos vienen de la base de datos local o de la API de Retrofit.

ViewModel: Gestiona el estado de la UI y se comunica con el Repositorio.

View (Activities/Fragments): Solo se encarga de mostrar los datos y capturar eventos del usuario.

2. Base de Datos Local con Room

Implementaremos una base de datos para almacenar el historial de transacciones y el perfil del usuario, permitiendo el acceso offline.

Entidad de Transacción:

```
@Entity(tableName = "transactions_table")
public class TransactionEntity {
    @PrimaryKey(autoGenerate = true)
    private int id;
    private String description;
    private double amount;
    private String date;
}
```

DAO (Data Access Object):

```
@Dao
public interface TransactionDao {
    @Insert
    void insert(TransactionEntity transaction);
    @Query("SELECT * FROM transactions_table ORDER BY date DESC")
    LiveData<List<TransactionEntity>> getAllTransactions();
}
```

3. Comunicación con API REST mediante Retrofit

Configuraremos Retrofit para conectarnos al backend y sincronizar las transacciones virtuales.

Interfaz de la API:

```
public interface WalletApiService {
    @GET("transactions")
    Call<List<TransactionResponse>> getTransactions();
    @POST("transactions")
    Call<TransactionResponse> postTransaction(@Body TransactionRequest request);
}
```

Cliente Retrofit:

```
public class RetrofitClient {
    private static Retrofit retrofit;
    private static final String BASE_URL = "https://api.alkewallet.com/";
    public static Retrofit getRetrofitInstance() {
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                    .baseUrl(BASE_URL)
                    .addConverterFactory(GsonConverterFactory.create())
                    .build();
        }
        return retrofit;
    }
}
```

4. Implementación del ViewModel

El ViewModel expondrá los datos a la vista utilizando LiveData.

```
public class WalletViewModel extends AndroidViewModel {
    private TransactionRepository repository;
    private LiveData<List<TransactionEntity>> allTransactions;
    public WalletViewModel(@NonNull Application application) {
        super(application);
        repository = new TransactionRepository(application);
        allTransactions = repository.getAllTransactions();
    }

    public LiveData<List<TransactionEntity>> getAllTransactions() {
        return allTransactions;
    }
    public void makeTransaction(TransactionRequest request) {
        repository.sendTransactionApi(request);
    }
}
```

Documentación Técnica: Proyecto Alke Wallet (Módulo 6)

1. Descripción de la Arquitectura

Para este proyecto se ha implementado el patrón de diseño MVVM (Model-View-ViewModel). Esta elección se justifica por la necesidad de separar la lógica de negocio de la interfaz de usuario, facilitando la mantenibilidad, el testeo y la escalabilidad de la billetera digital.

Componentes y Responsabilidades:

Model (Capa de Datos): Gestiona el acceso a los datos. Incluye las entidades de Room para la persistencia local y las interfaces de Retrofit para la comunicación con la API REST.

View (Actividades/Fragments):

Su única responsabilidad es mostrar la información al usuario y observar los cambios en el ViewModel a través de LiveData. No contiene lógica de negocio.

ViewModel:

Actúa como puente. Solicita datos al Repositorio y los expone a la Vista. Sobrevive a los cambios de configuración (como la rotación de pantalla), manteniendo la integridad de los datos de la sesión**.**

Repository:

Funciona como una "fuente única de verdad". Decide si los datos deben recuperarse de la caché local (Room) o sincronizarse desde la nube (Retrofit).

2. Gestión de Datos Local (Room)

Se implementó Room para garantizar que el usuario pueda consultar su saldo e historial de transacciones incluso sin conexión a internet.

Entidades:

UserEntity (perfil) y TransactionEntity (movimientos).

DAO (Data Access Object):

Define las consultas SQL necesarias para insertar transacciones y obtener el historial ordenado por fecha de forma asíncrona.

Database:

Clase abstracta que extiende de RoomDatabase, implementando el patrón Singleton para evitar múltiples instancias de la base de datos que podrían agotar recursos**.**

3. Integración de API Externa (Retrofit)

La conectividad con el backend de la billetera se gestiona mediante Retrofit, permitiendo que las transacciones virtuales sean persistentes en un servidor central.

Interfaz del Servicio:

Define los endpoints de tipo @GET para el historial y @POST para ejecutar nuevos envíos de dinero.

Manejo de Errores:

Se implementó una lógica de gestión de respuestas para capturar códigos de error (como 401 Unauthorized o 500 Server Error), notificando al usuario mediante mensajes claros en la UI.

Conversión de Datos:

Se utiliza GSON para la serialización y deserialización automática de los objetos JSON**.**

4. Flujo de Usuario y Validaciones

Registro e Inicio de Sesión:

Se diseñaron pantallas con validaciones en el lado del cliente antes de enviar datos al servidor:

Email:

Validación mediante expresiones regulares (Regex).

Contraseña:

Integración de la lógica de fortaleza (Weak/Medium/Strong) para incentivar el uso de credenciales seguras.

Historial y Transacciones:

El usuario puede visualizar sus movimientos en un RecyclerView que se actualiza automáticamente gracias a la observación de LiveData en el ViewModel.

5. Pruebas y Calidad (Unit Testing)

Se incluyeron pruebas unitarias utilizando JUnit y Mockito para asegurar la calidad del software:

ViewModel Tests:

Verificación de que el ViewModel procesa correctamente los montos de las transacciones antes de enviarlos.

Room Tests:

Pruebas sobre los DAOs para asegurar que la inserción y recuperación de datos locales sea íntegra.

6. Conclusiones y Decisiones de Diseño

Uso de LiveData:

Se prefirió sobre otros métodos para evitar fugas de memoria, ya que los observadores están vinculados al ciclo de vida de la actividad.

Inyección de Dependencias:

Se estructuró el código para que sea fácil inyectar un repositorio falso (mock) durante las pruebas de integración.